

It's easy to find that it's optimal to greedily select the shortest possible prefix each time. Therefore, only checking whether a prefix is satisfied should be done.

Firstly, to keep the sets of each person being the same, we can use an array of counter counting how many times each number appears, and another counter counting how many counters are neither 0 nor M .

Secondly, to keep the set if a set of leaves, one method is to check whether the remaining set is connected, which can $O(1)$ checked by recording the result of number of nodes minus number of edges. Another method is to directly maintain the count of deleted non-leaf nodes and jump to the parent node when deleting to a leaf.

The time complexity is $O(\sum nm)$.



Subtask 1

Let $dp[i][j]$ be the minimum expenditure to arrive in Planet i at time j . Run Dijkstra's Algorithm to obtain the answer. Time complexity is $O(MT \log N)$, where $T = \max_{1 \leq i \leq M} b_i$.

Subtask 2

Method 1: Prefix Min

Let's define $dist[i]$ as the minimum expenditure to finish the train route i . Sort all train routes by increasing b_i and we calculate $dist$ in that order. $dist[i]$ can be transitioned from the minimum $dist$ value of any other train route which arrives in Planet x_i not later than time a_i . To obtain that value, we can create N arrays, each containing the $dist$ values of all train routes ending at a particular planet. Within each array, train routes can again be sorted by increasing b_i , so each query corresponds to a prefix min on $dist$ values in the array. We can simply insert each newly calculated $dist$ value at the back of the array for Planet y_i and calculate the prefix min at the same time (note that prefix min at earlier positions remains unchanged). In each query, we can use a binary search to find the corresponding prefix to consider. Time complexity is $O(M \log M)$.

Method 2: Dijkstra

Construct a new graph. Each node is now defined as $[current\ planet, current\ time]$. For each edge in the original graph, we add an edge with the same weight (i.e., c_i) from node $[x_i, a_i]$ to node $[y_i, b_i]$ in the new graph. Hence, we have at most $2m$ nodes. Furthermore, for all nodes of the same $current\ planet$, we add free (i.e., weight = 0) edges such that they form a directed linear graph of increasing $current\ time$. For example, if we have train routes arriving/departing in Planet 3 at time 1, 2, 5 only, then we add an edge from $[3, 1]$ to $[3, 2]$ and another edge from $[3, 2]$ to $[3, 5]$. We can then run Dijkstra on the new graph, where the starting node is $[0, 0]$ and the answer will be the minimum value among shortest distances to any nodes associated with Planet $N - 1$. The time complexity is $O((M + N) \cdot \log M)$.

Subtask 3: meals are disjoint.

Method 2 from Subtask 2 can be improved by modifying the states used in the Dijkstra's Algorithm. For each node in the new graph mentioned above, we introduce another $0/1$ state to represent whether the meal covering `current time` has been taken. This meal should be unique under this subtask. Particularly, `1` is assigned if no meal covers the time. The greedy way to take meals is (1) if we are on a train, eat the free meal, (2) otherwise we procrastinate the meal. (2) works as all paid meals must be taken between train routes, so the exact time to take the meal doesn't matter (we would be in the same planet throughout the possible time interval of the meal anyway), meaning that we could just take the meal as late as possible. From here, we can figure out how to do the transition. For edges corresponding to train routes, we always transit to state `1` since we can always have meals for free on the train. For free edges, we transit to `1` if no meal covers the `current time` of the end node and otherwise transit to `0`. Note that a cost of transition is generated from all meals that happen strictly between the two times at the endpoints; If we are transiting from state 0, there's one extra meal to count if the meal covering the `current time` of the starting node does not last till the `current time` of the end node. Time complexity is $O(M \log(MW))$.

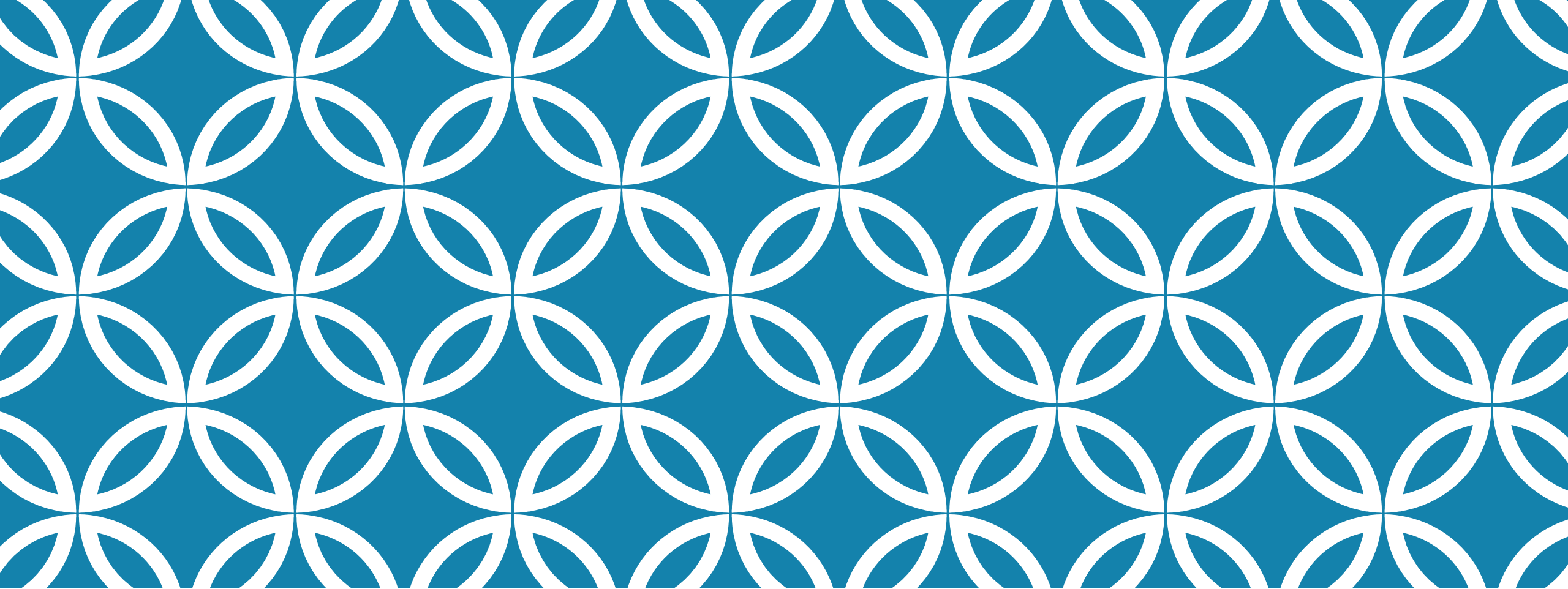
Full solution

Process all train routes in increasing a_i . For each train route i , we store a dynamic `dist` value: the minimum expenditure to finish i and finish all meals that have to be taken before a_{cur} , where cur is the index of train route that's currently being processed. If we store `dist` in a similar way as described in **Method 1 from Subtask 2** (i.e., routes ending at the same planet are stored in one array by increasing b_i), `dist` values can be again obtained from a prefix min. In fact, we can avoid finding a prefix by only appending `dist` values of train routes t which satisfy $b_t \leq a_{cur}$. Appending `dist` values can be implemented with a priority queue over processed trains, sorted by b_i . Then, `dist` values can be obtained from the minimum across the array storing train routes ending at x_{cur} plus c_{cur} .

The next tricky part is how to update `dist` values using meals. Meals that should be counted towards `dist` values satisfy $r_j < a_{cur}$. Since train routes are processed by increasing a_i , we should sort meals by increasing r_j and use them to update all `dist` when appropriate. Note that each meal updates a particular prefix for each `dist` array -- this can't be implemented

efficiently. To optimize this, we treat each array as a monotonic queue, where the `dist` values must be strictly increasing, as meals always worsen a prefix. Then, to obtain `dist` values, we only need to query the first value from the corresponding monotonic queue. But how to handle the updates? For each route p in Planet u 's monotonic queue, we can predict when its `dist` value will exceed that of the next route q : Let val_p and val_q be their **initial** `dist` values (i.e., not updated by meals strictly after the train routes), and q will be better than p once $K = \lceil \frac{val_q - val_p}{c_u} \rceil$ meals that satisfy $b_p < l_j \leq b_q$ are processed. The exact meal where the overtaking happen can be identified by finding the K th largest r_j over a range on meals sorted by l_j . In fact, instead of finding the K th largest r_j , it might be easier to just give each meal an index based on increasing r_j and find the K th largest index. Range K th can be implemented by "walking" on two persistent segment trees, where each persistent segment tree corresponds to a prefix in meals, and its node stores the occurrences of indices in a certain range. To find the K th largest in the range $[L, R]$, We start with two pointers from the roots which corresponds to the prefix of $L - 1$ and R respectively. The two pointers should always correspond to the same range in indices, and we can calculate the occurrences of indices in the shared range over $[L, R]$ by taking the difference between their corresponding values stored. This allows us to determine whether to go left or right (simultaneously for both pointers) when travelling to a lower layer, and the "walking" stops when the two pointers both arrive at nodes for just one index.

Time complexity is $O(M \log(MW))$, possibly with considerable constant terms.



MAGIC SHOW SOLUTION

STATEMENT

Alice decide an inter n , between 2 and 5000. Grader then gives Alice an integer X , and Alice should give grader a tree with n vertices.

Then, grader deletes at most $\left\lceil \frac{n}{2} \right\rceil - 1$ vertices, and gives Bob the forest. Bob should tells grader what X is.

$$1 \leq X \leq 10^{18}.$$

SUBTASK 1

$X \leq 5000$

Alice can just make a star graph whose centroid is X . Then Bob can identify what the centroid was, and can guess X correctly.

SUBTASK 2

We can convert this subtask to transport two integers x, y , between 1 and 5000. First, connect all the vertices in 1 and 1666 to x . Then, connect all the vertices between 1667 and 3333 to y . Finally, connect all vertices between 3334 and 5000 to $x+y \bmod 5000$.

If there is a cycle in this graph, then delete any edges in the cycle, until the graph has no cycle.

After that, connect components in the graph, in any ways.

SUBTASK 2

Then we will receive the graph. There would be at least two vertices with more than 5 degree(very roughly), because by deleting at most 2499 edges, it's impossible to make two vertices among $x, y, x+y$, to have degree lesser than 5.

Thus, we can know two values between $x, y, x+y \bmod 5000$, and know which value corresponds to them.

Then, we can restore x and y , then we can guess the X in original problem, correctly.

FULLTASK

The main idea is to convert original problem to transport five values, x_1, x_2, x_3, x_4, x_5 , between 1 and 4096.

Claim. consider n values, called $x_1, x_2, \dots, x_n$, between 0 and 4095. We can make $2^n - 1$ values between 0 and 4095 so that we can restore $x_1, \dots, x_n$ when at most half of $2^n - 1$ values can be deleted.

WHY?

We can make $2^n - 1$ values as following:

Consider $2^n - 1$ sets except empty set, which has elements as $x_1, \dots, x_n$.

Then, XOR all the values in the set.

For example, if $x_1=1, x_2=4$, then we will get $\{x_1\} \rightarrow 1, \{x_2\} \rightarrow 4, \{x_1, x_2\} \rightarrow 5$.

WHY?

Because the value that corresponds to empty set would be zero, we can consider we have 2^n value, which corresponds all possible combinations of $x_1, \dots, x_n$.

Because only half of $2^n - 1$ values can be deleted, so there would remain $2^{n-1} + 1$ values, including 0 from empty set.

Consider x_i . for every possible 2^{n-1} set which doesn't contain x_i , There exists S s.t. S remains, and $S \cup \{x_i\}$ remains.

Because we know XOR of two sets, we can get x_i by just XORing two values.



WHY?

So we can restore all the values, and Claim could be proved.

FULLTASK

So, it's enough to transport 31 values (we will call them a_k , where $1 \leq k \leq 31$.) between 1 and 4096 to solve entire problem.

To do that, first, we will set $n=4991$. I will explain why this setting is required.

for each k , connect vertices between $160(k-1)+1$ to $160k$ to $(k \text{ th value})\text{th}$ vertex.

FULLTASK

So, it's enough to transport 31 values between 1 and 4096 to solve entire problem.

To do that, first, we will set $n=4991$. I will explain why this setting is required.

for each k , connect vertices between $161(k-1)+1$ to $161k$ to $(k \text{ th value})\text{th}$ vertex.

If there exist some cycles, then delete any edges in the cycle, until there is no cycle.

What we can notice is that, the original graph was functional graph, so in this procedure, for each vertices, degree decreases at most 1.

FULLTASK

Then, we will get forests with some components, which has size not smaller than 160 .

Thus, we can select distinct leaves from each components, and connect them moderately so that the vertices we selected has degree lesser than 3 .

So, it's the end of Alice's tree construction.

FULLTASK

Then, How can bob know 16 values among 31 original values?

In The tree bob have received, for $1 \leq k \leq 31$, we will call k good if there exists some vertex V , so that in $[161(k-1)+1, 161k]$, there exists at least 2 vertices adjacent to V .

Before transforming, every k is good, and there are at least 159 vertices adjacent to V .

What we can observe is that, for each k , V is unique.

FULLTASK

There can be at most 30 edges, which connects first 31 values.

So, erasing 2495 edges, it can decrease number of adjacent vertices to a_k , in total, at most 2525.

For a k , at least 158 vertices should be erased to make k not good (Because, for each a_k , they will have degree at least 159 (=160 (the least degree a_k has in the original functional graph) - 1 (when we delete some cycle)))

So, deleting at most 2525 values, we can make at most $\left\lceil \frac{2525}{158} \right\rceil = 15$ k 's to be not good.

FULLTASK

So, there exists 16 good k 's between 1 and 31.

Then, we can just apply the Claim, and then we can decode X . So, the entire problem could be solved.

WHY $N=4991$ NEEDED?

if we set $n=5000$, then 2499 edges can be deleted, and $2529/158$ is bigger than 16.

In more detail. increase of n is asymptotically benefit for solving problem. But, this effect occurs because as n increases, then denominator increases “sometimes”. So, if n decreases, and denominator doesn’t decreases, then numerator decreases, so it can be benefit, sometimes.